

Team 11

Voting System

Software Design Document

Name (s): Manan Chaturvedi, Andy Dang, Michael Dunn, Mark Tsai

Date: (03/02/2025)

TABLE OF CONTENTS

1.	INTRODUCTION	2
1.1	Purpose	2
1.2	Scope	2
1.3	Overview	2
1.4	Reference Material	2
1.5	Definitions and Acronyms	2
2.	SYSTEM OVERVIEW	3
3.	SYSTEM ARCHITECTURE	4
3.1	Architectural Design	4
3.2	Decomposition Description	5
3.3	Design Rationale	7
4.	DATA DESIGN	7
4.1	Data Description	7
4.2	Data Dictionary	8
5.	COMPONENT DESIGN	10
6.	HUMAN INTERFACE DESIGN	13
6.1	Overview of User Interface	13
6.2	Screen Images	13
6.3	Screen Objects and Actions	14
7.	REQUIREMENTS MATRIX	14
8.	APPENDICES	17

1. INTRODUCTION

1.1 Purpose

This software design document describes the architecture and system design of the voting system. The intended audiences are project managers, developers, and anyone else who will be involved in implementing or maintaining the software system within its life cycle.

1.2 Scope

The software is designed to perform plurality or single transferable vote (STV) voting. This project aims to provide a trustworthy and testable voting system for people who want to hold an election, study the voting processes and results, or perform further development with this codebase.

1.3 Overview

The document is a Software Design Document that describes the outline of the architecture and design specifications of the voting system. The document comprises eight sections: introduction, system overview, system architecture, data design, component design, human interface design, requirement matrix, and appendices.

1.4 Reference Material

There is no reference material in this document.

1.5 Definitions and Acronyms

Plurality	The number of votes cast for a candidate who receives more than any other but does not receive an absolute majority.
STV	Single transferable vote. A proportional representation voting method where voters rank candidates in order of preference. A candidate needs to reach a quota of votes to be elected. If a candidate exceeds the quota, their extra votes are transferred to voters' next preferences. If no one meets the quota, the candidate with the fewest votes is eliminated, and their votes are transferred. This process continues until all seats are filled.

2. SYSTEM OVERVIEW

This program serves as a simulation to run either a STV or Plurality election given a set of data of pre-chosen ballots (inputted through a command line as text, stored as a .csv file), along with the total number of seats, the type of election to be run, and - in the event of testing the program's validity - a shuffle flag. The shuffle flag will not be toggled by an election official, though in the event of the shuffle flag being toggled, the integrity of the election's results should remain. After the input is given, the system will first parse all of the information from the CSV File and create a Ballots class. This will locate the number of candidates and their names as strings, along with the total number of candidates as an integer and formatting for all the ballots as a vector of integers. The unique identifier of each ballot is their index within this vector. Also, if the shuffle variable is true, the shuffle() function will shuffle the ballots and store the state of the variable as a boolean. The Ballots class created will be a parameter for either an STV or Plurality class, depending on the user's input. The classes themselves will share similar information about the Ballots class, along with an integer containing the number of seats and an instantiated vector of Candidates - where Candidate is its own object - created from all the candidates within the Ballots class. There will also be a vector of Candidates for winners and losers, but they will not have any value until the election progresses. The Candidate class itself will contain the name of the candidate as a string, along with the total number of ballots as an integer - initialized as 0 - and the specific ballots assigned to them as a vector of integers. Then, runElection() is run for either the STV or Plurality election, and each will have its own implementation. The STV class will perform runElection() by going through rounds of the entire set of ballots to determine the winners and losers based on a calculated Droop Quota from the number of seats and ballots. In contrast, the Plurality version of runElection() will simply go through the entire set of ballots once to determine the winners and losers. However, during the STV's runElection(), each round will be documented with a vector of strings to display onto an audit file (formatted as a .txt file, of which the name is input by the user at the end of runElection()). After runElection() has finished executing, there will be a function displayElectionDetails(), which will display the election results within the format of each election type - STV will have information relating to the order of the winners and losers, while Plurality will contain the percentage of votes given to each candidate, but both will share the same information about the number of seats, ballots, candidates, and the type of election ran. This information will be printed to the user's command line interface so they can clearly see the election information, allowing a simple method of determining an election official within the scope of an election official's capabilities.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

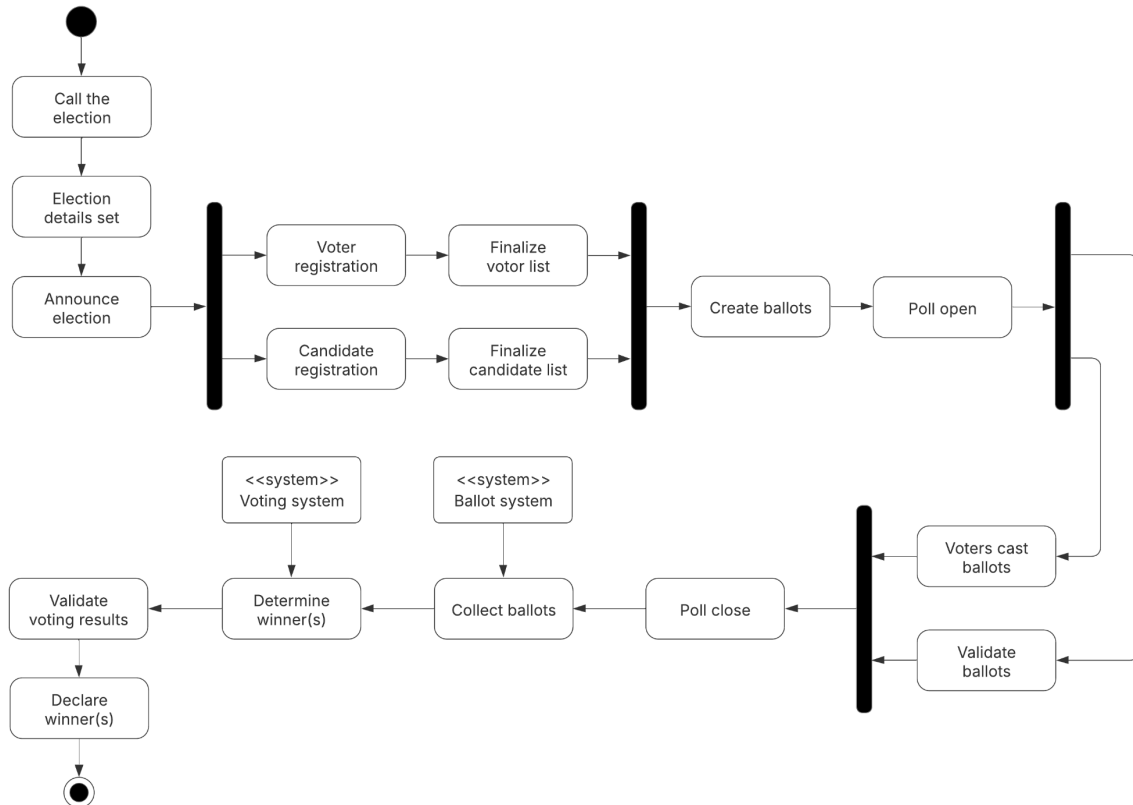


Figure 1: Activity diagram of the voting system

Figure 1 is a high-level overview of the interaction between processes within the voting system for both STV and plurality elections, starting from the scheduling of an election. To start, an election is called, and election details are specified and announced before voters and candidates register to be a part of the election. Once voter and candidate lists are finalized, ballots are created, and the polls are opened, allowing ballots to be cast and validated. Finally, polls are closed, and ballots are collected using the ballot system, and the voting system is used to determine election winners. Depending on which algorithm is selected by the election official/tester, the winner of the election will be determined using the STV or plurality algorithm.

3.2 Decomposition Description

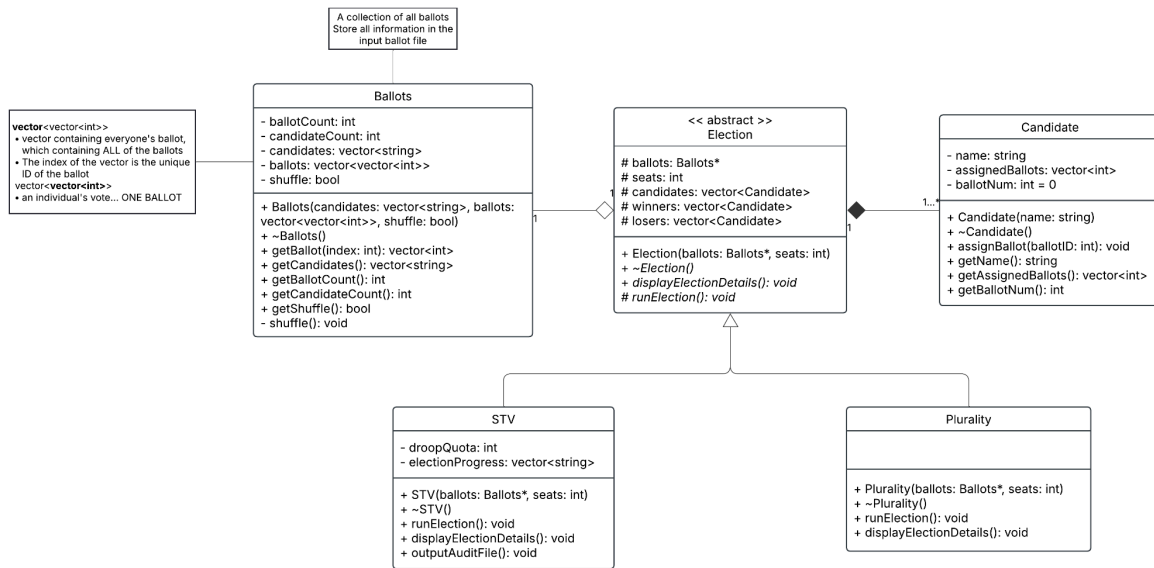


Figure 2: Modular UML class diagram of the voting system

Figure 2 above displays the relationships between classes within the voting system. In the center, the abstract Election class includes election information, including the shuffle flag, number of seats being elected, candidate information (see Section 4 for data structure information), and winners/losers of the election. There is also a pointer to a “Ballots” object that contains the voters’ candidate selections (see Section 4). The STV and Plurality classes inherit from the abstract Election class, each containing functions for running their respective election types. This allows both elections to reuse common functionality within the Election class. The STV method contains member variables “droopQuota” to keep track of the current election’s Droop quota and an election progress vector that contains information about the STV election as it progresses (see Section 4). The Candidate class allows for objects to be created that represent each candidate. Each candidate object stores a vector containing the ballot IDs of each ballot the candidate has received.

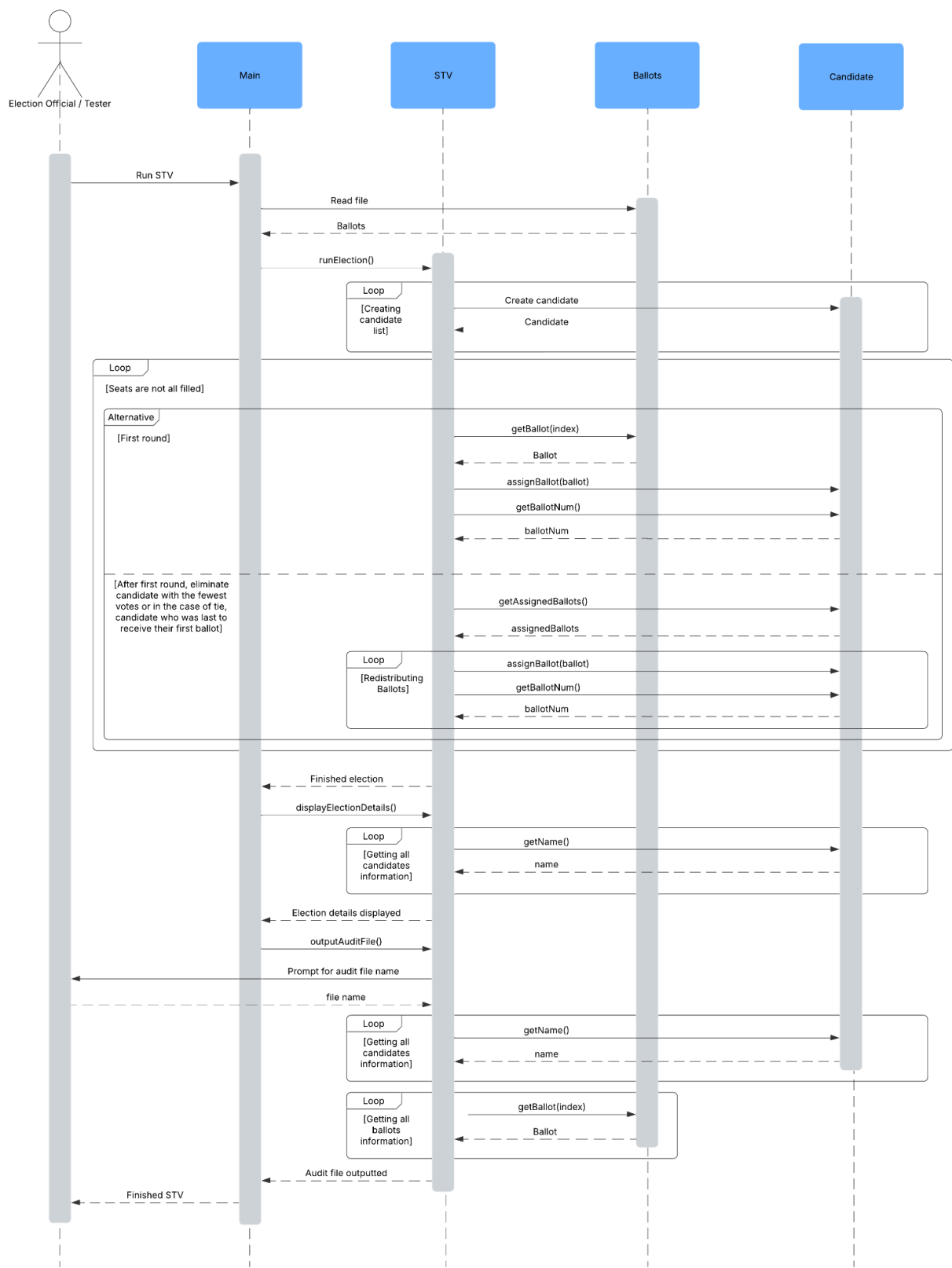


Figure 3: Sequence diagram for the running of the STV election, starting from the moment of receiving the ballots into the system from the CSV file.

Figure 3 outlines the interaction between the Main, STV, Ballots, and Candidate classes during an STV election. Starting from the moment ballots are received into the system from the user-submitted file; Figure 3 shows the Ballots class reading the file containing ballots and returning a Ballots object. Then, the runElection() function is run by the STV class, leading to a loop that creates a vector list of Candidate objects. Once the candidate list is created, the STV algorithm is started. The outer loop ran within the system checks whether all of the election's seats have been filled, and if not, continues to the rest of the STV algorithm's steps. Following the first round of the election, if the total number of seats has not been elected by the system, the ballots of the candidate with the fewest votes are redistributed to the voter's next choice. Finally, once all the seats have been filled, the election finishes, and the displayElectionDetails() function is called. This loops through the candidate list and returns the candidate information. A file name for the audit file is then prompted, and the output follows two loops that retrieve ballot and candidate information, respectively.

3.3 Design Rationale

To reuse the election components shared by STV and Plurality elections (such as candidates, seats, winners, etc.), we designed each election type as a realization relationship between a shared abstract Election class. This architecture maximizes code reuse and emphasizes the distinction between the two election algorithms. Another design choice we made was to create separate classes for ballot and candidate information. The individual classes enforce the single responsibility principle by removing data storage as a responsibility of the election class. Finally, we included the CSV file reading and translation of the CSV file into a Ballots object in the main class. We initially considered including this functionality in the Election or Ballots class. Still, this implementation ensures that the Ballots class is only required to store ballot information, and the Election class is only required to run elections. This leaves the file reading separate and enforces the single responsibility principle.

4. DATA DESIGN

4.1 Data Description

In our voting system, the voting information of each election participant is input by an election official/tester in the form of a CSV file. Once the file has been successfully opened and read, the main class will read each line of the CSV file (each line corresponds to one ballot) and create a Ballots object with a 2-dimensional vector that contains all ballots. In the 2-dimensional "ballots" vector, each row corresponds to an individual ballot, and each column contains a candidate option, leaving us with an $m \times n$ vector where m is the number of votes and n is the number of candidates. Within the Candidate class, a vector contains the IDs of ballots that were distributed to that candidate during the election. This facilitates the redistribution of ballots if a candidate does not reach the Droop quota during an STV election.

In the Election class, a vector of Candidate objects containing both winners and losers of the election is stored. This allows for easy access to winning and losing Candidate information during the display of final election results and the creation of audits. In the STV election class, a vector containing strings is stored to keep track of vote distribution for audit output. Overall, we chose to use vectors as the primary data structure for our voting system due to their flexible size and ease of implementation.

4.2 Data Dictionary

Ballots:

Attributes:

- ballotCount (int): integer used to store the number of ballots input by the user. Included in election details and useful for calculating the Droop quota.
- candidateCount (int): integer used to store the number of candidates that voters picked from. Also included in election details.
- candidates (vector<string>): the string containing candidate names. When the CSV file is read for the first time, the first line containing candidate names is stored here.
- ballots (vector<vector<int>>): a 2-dimensional vector for storing each voter's ballot and candidate choices. See Section 4.1 for vector details.
- shuffle (bool): the boolean that stores the election official/tester's input on whether the votes should be shuffled. A value of True indicates the votes need to be shuffled.

Methods:

- Ballots(candidates: vector<string>, ballots: vector<vector<int>>, shuffle: bool): creates a Ballots object to contain all ballot information.
- ~Ballots(): Ballots destructor used for memory cleanup after an election is run.
- getBallots(index: int): returns the ballot at index "index" as a vector<int>.
- getCandidates(): returns the vector<string> containing all of the candidate's names.
- getBallotCount(): returns an integer containing the number of ballots.
- getCandidateCount(): returns an integer containing the number of candidate options.
- getShuffle(): returns the boolean containing the shuffle flag.
- shuffle(): no return type, shuffles the outer vector of the vector<vector<int>> containing all of the votes, shuffling all of the ballots into a random order.

Candidate:

Attributes:

- name (int): the name of the candidate represented by the Candidate object.
- assignedBallots (vector<int>): the vector of ballots IDs that represent the ballots assigned to the candidate. See Section 4.1 for assigned ballot vector details.
- ballotNum (int): the number of ballots for the candidate.

Methods:

-
- `Candidate(name: string)`: the `Candidate`'s constructor creates a `Candidate` object representing one of the election's candidates.
 - `~Candidate()`: `Candidate` destructor used for memory cleanup after an election is run or after a candidate is out of the election.
 - `assignBallot(ballot: vector<int>)`: each vote that the candidate receives is added to their `assignedBallots` vector with this method.
 - `getName()`: returns a string containing the candidate's name.
 - `getAssignedBallots()`: returns the `vector<int>` of the ballots assigned to the candidate.
 - `getBallotNum()`: returns an integer representing the number of ballots assigned to the candidate.

Election:

Attributes:

- `ballots (Ballots*)`: a pointer to the `Ballots` object containing all of the ballots. See Section 4.1 for more information about ballot storage.
- `seats (int)`: an integer containing the number of seats that are being elected during the election.
- `candidates (vector<Candidate>)`: a vector containing `Candidate` objects, used to store information about and keep track of votes for each candidate.
- `winners (vector<Candidate>)`: a vector containing `Candidate` objects, used to keep track of winning candidates for use in creating audits and displaying election results.
- `losers (vector<Candidate>)`: a vector containing `Candidate` objects, used to keep track of losing candidates to create audits and display election results.

Methods:

- `Election(ballots: Ballots*, seats: int)`: the `Election` constructor takes in all ballots and the number of seats being elected, and creates an `Election` object.
- `~Election()`: `Election` destructor for memory cleanup after an election is run.
- `displayElectionDetails()`: prints out the type of algorithm used in the election, the number of ballots, the number of seats filled, the number of candidates, and the winning candidates. This method is virtual and overridden by each election algorithm to include additional functionality.
- `runElection()`: a virtual function overridden by each election algorithm depending on the algorithm selected by the election official/tester.

Plurality:

Attributes:

- No attributes are stored within the `Plurality` class.

Methods:

- `Plurality(ballots: Ballots*, seats: int)`: the `Plurality` constructor takes in all ballots

and the number of seats elected and creates an election with the election functionality of a plurality election.

- ~Plurality(): Plurality destructor for memory cleanup after an election is run.
- runElection(): overrides the runElection() method in the Election class and implements additional functionality specific to plurality elections.
- displayElectionResults(): overrides the displayElectionResults() method in the Election class and implements additional functionality specific to plurality elections.

STV:

Attributes:

- droopQuota (int): stores the calculated Droop quota.
- electionProgress (vector<string>): a vector of strings to store the progress of the election throughout the election process. This contains information about when candidates reach Droop quota or when candidates are moved to the winners/losers list.

Methods:

- STV(ballots: Ballots*, seats: int): the STV constructor takes in all ballots and the number of seats elected and creates an election with the election functionality of an STV election.
- ~STV(): STV destructor for memory cleanup after an election is run.
- runElection(): overrides the runElection() method in the Election class and implements additional functionality specific to STV elections.
- displayElectionResults(): overrides the displayElectionResults() method in the Election class and implements additional functionality specific to STV elections.

5. COMPONENT DESIGN

Election:

- void runElection() = 0 *// pure virtual*
- void displayElectionDetails() = 0 *// pure virtual*

Ballots:

- vector<int> getBallot(int index) {
 return ballots[index] *// ballots is a vector of a vector of ints*
}
- vector<string> getCandidates() {
 return candidates *// candidates is a vector of strings containing candidate names*
}
- int getBallotCount() {
 return ballot count *// ballot count is the amount of ballots*
}

```

    }

-   int getCandidateCount() {
        return candidate count // candidate count is the amount of candidates
    }

-   bool getShuffle() {
        return shuffle // shuffle is a bool
    }

-   void shuffle() {
        call a shuffle function on ballots // ballots is a vector of a vector of ints
    }

```

Candidates:

```

-   void assignBallot(int ballot id) {
        add ballot id to assigned ballots // assigned ballots is a vector of ints
    }

-   string getName() {
        return name // name is a string containing the name of a candidate
    }

-   vector<int> getAssignedBallots() {
        return assigned ballots // assigned ballots is a vector of ints
    }

-   int getBallotNum() {
        return ballot number // ballot number is the amount of ballots assigned to the candidate
    }

```

Plurality:

```

-   void runElection() {
        for each ballot in ballots:
            find the index of that contains the integer 1 inside the ballots vector
            increment ballot number inside the vector of Candidate objects at index
        for each seat in seats:
            determine the Candidate object with the highest ballot number
            if amount of candidates with highest ballot number is greater than 1
                call a random number generator between 1 and number of
                winning candidates to decide winning candidate
        add winning candidates to win list
        add losing candidates to lose list
        return
        // ballots is a pointer to a Ballots object
    }

```

```

    // ballot number is the amount of ballots assigned to the candidate
    // candidates is a vector of Candidate objects
}

```

```

- void displayElectionDetails() {
    print "Plurality Election"
    print number of seats, number of ballots, number of candidates
    print "Winner(s)"
    print election winners
    print "Loser(s)"
    print election losers
    print all candidates with the percentage of votes they received
}

```

STV:

```

- void runElection() {
    if shuffle is true:
        call shuffleBallots() on ballots
    calculate Droop quota and assign it to droopQuota
    while not on last ballot of ballot list:
        get current ballot in ballot list
        while current ballot is not assigned:
            if current ballot's priority count is greater than max rank:
                remove current ballot from ballot list
            if current ballot's priority candidate not is on losers list:
                assign ballot to corresponding candidate
                break while loop
            else:
                increment current ballot's priority count
        if candidate votes is greater than droop:
            add candidate to end of winner list
            remove all ballots assigned to candidate from ballot list
        increment ballot
    check non-winner and non-loser candidates for number of assigned ballots
    determine candidate with least number of ballots
    if there are multiple candidates:
        determine last candidate to receive their first vote
    add candidate to top of lose list
    ballot list is assigned to the ballots of the losing candidate
    if there are no ballots in ballot list:
        return
    else:
        repeat "while not on last ballot of ballot list"
}

```

```

// ballots is a pointer to a Ballots object
// droopQuota is an int signifying the amount of votes needed to win the election

```

```
// ballot list is a list of ballots that are valid for counting
// priority count is an int signifying which candidate the ballot is assigned to
// candidate votes is the # of votes of the most recent candidate to get a ballot
}

- void displayElectionDetails() {
    print "STV Election"
    print number of seats, number of ballots, number of candidates
    print "Winner(s)"
    print election winners
    print "Loser(s)"
    print election losers
    print ranking of candidates from biggest winner to biggest loser
}
```

6. HUMAN INTERFACE DESIGN

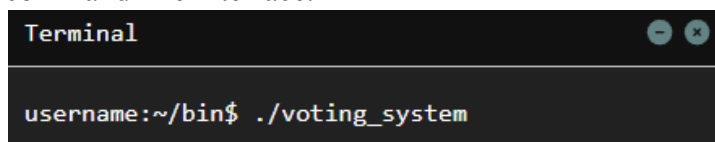
6.1 Overview of User Interface

The only software component the user interface requires is a terminal. After the program starts with the command, it will provide instructions to guide the user through the election process. For instance, the system will first ask the user to input the name of the CSV file containing ballot information, and the user will input the file name before proceeding to the next step. If a user inputs an incorrect file name, the system will notify the user of its inability to locate the file and ask the user to re-input it. If the file name is valid, the system will inform the user that the file is successfully opened. Following is the visualized user interface of the example.

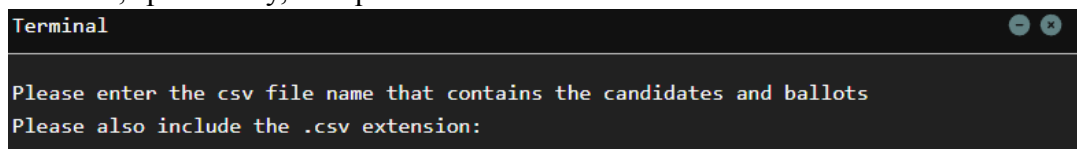
6.2 Screen Images

The following screen images are showing the interface example mentioned in section 6.1.

1. The program is started by typing “./voting_system” within the operating system’s command line interface.

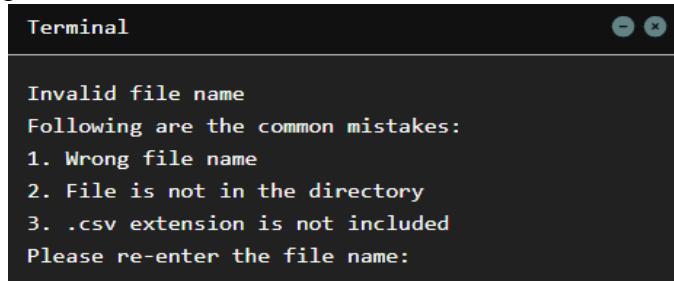


2. After the software starts, the system will first ask the user to input the name of the CSV file; specifically, it requires that the .csv extension be included.



3. If the file name is invalid, the system prompts the user with an error message and

possible causes of the error.



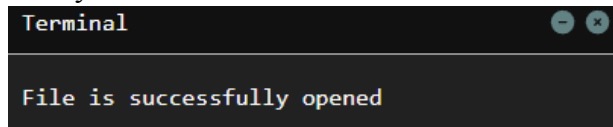
```

Terminal

Invalid file name
Following are the common mistakes:
1. Wrong file name
2. File is not in the directory
3. .csv extension is not included
Please re-enter the file name:

```

4. The system informs the user that the file name is valid and that the file is opened.



```

Terminal

File is successfully opened

```

6.3 Screen Objects and Actions

The screen object is the text in the terminal, including instructions, messages, and inputs. Any action associated with the text would be the software system's response to the user's input.

7. REQUIREMENTS MATRIX

System Component	ID	Use Case
- This use case will be done within Main. - The shuffle variable is set to false when the software is started with the shuffle-off flag for testing; otherwise, it is set to true. - shuffle behavior is performed by the shuffle() function within the Ballots class.	UC_001	Take the shuffle-off flag
- This use case will be done within Main. - The user will be prompted with the name of the CSV file using the command line terminal interface using std::cin.	UC_002	Open the CSV file
- This use case will be done within Main. - Done within the command line terminal interface as an input prompt using std::cin.	UC_003	Prompt the user for the number of seats being elected
- This use case will be done within Main. - Done within the command line terminal interface as an input prompt using std::cin.	UC_004	Prompt user for the algorithm to be used in the election process.
- This use case will be done within Main.	UC_005	Load information

- The chosen data structure will be Ballots and contain the ballotCount, candidateCount, candidates, ballots, and a shuffle variable (gotten from UC_001).		in the opened CSV file to the data structure
- This use case will be done within Main. - Done within the Ballots data structure created from the process of loading in the CSV (ballotCount and candidateCount).	UC_006	Determine the number of ballots and number of candidates
- This use case will be done ONLY if the inputted algorithm from UC_004 is STV within the STV class. - This is calculated when runElection() is performed after the processing of the file.	UC_007	Determining the Droop quota
- This use case will be done ONLY within the STV class. - Prompting the user for the STV audit report file name will only be done AFTER the algorithm is confirmed to be STV. - This will be prompted within the STV algorithm AFTER the program has finished running the program; once the election has finished running, the details of the election will be stored within the string electionProgress.	UC_008	Prompting the STV audit report file name
- This use case will be done ONLY within the STV class. - Outputting the STV audit report file will be done ONLY if the chosen algorithm is STV. - The outputting of the file will be done AFTER the program has finished running. The name of the file will be determined through UC_008 and the output will be in a text file.	UC_009	Output STV audit report file
- This is ONLY run if the user chooses the plurality election and is run within the Plurality class's runElection() function. - The function serves to count the total number of ballots towards a created Candidate class for each candidate at the beginning. - The system will increment each Candidate class's ballotNum value by one with the function assignBallot(ballot: vector<int>) in the event a ballot votes for the candidate. The ballot is then not used for the rest of the election process. - This process goes through until every ballot has been	UC_010	Run the plurality algorithm

processed and assigned. - We sort the Candidates from the highest ballotNum value to the lowest and take the total number of seats as the top positions within the candidates vector and find the winners. - In the event of a tie, we will perform a coin flip (or roll a die in the event of there being multiple people) to fill in the seats.		
- This is ONLY run if the user chooses the STV election and is run within the STV class's runElection() function. - The function first calculates the Droop Quota using the total number of seats and the total number of ballots floor divided, then adding 1 to the total to get a value that, if any candidate gets this amount of votes, will automatically make them a winner. - Then, the STV algorithm will circle through each ballot in a round. Within the round, it will search for the ballot with the lowest number associated with a candidate that is NOT already declared a winner or loser through the known winners/losers variable within the STV class. Once it does find a valid match, then the candidate gets that ballot added to their list of ballots via the assignBallot() function. We will also check via the getAssignedBallots() check that the total number of ballots is less than the Droop Quota value. - If the candidate has NOT reached the Droop Quota, then we continue going through the ballots. - If the candidate HAS reached the Droop Quota, then it is added to the winner variable of STV so long as there is an available seat based on the seats variable of STV, otherwise we continue going through the ballots. Then, every ballot associated with the winner is removed from the total list of ballots available - this is kept track with the assigned ballots of the Candidate class. - Once all of the ballots have been dispersed, the person with the lowest number of ballots is eliminated - in the event of a tie, the person who receives the first ballot stays in the race, while the other is eliminated and added to the losers via the loser variable within STV. The first ballot obtained will be checked via the index within the assigned ballots of each tied Candidate.	UC_011	Run STV algorithm

- After the winners and losers of the round have been chosen, the algorithm continues with the ballots of the eliminated candidate being redispersed among the total ballots. This process continues until there are no ballots left to go through and there is a list of winners and losers. - It should be noted that after each round, the variable electionProgress will be updated about the winners and losers of the round, along with who voted for which Candidate. Each ballot recorded down will be recorded with its index serving as its unique identifier.		
- This is done within the displayElectionDetails class of both the STV and Plurality classes. - They will each display details of the election pertaining to who won and lost - and by what percentage of votes (Plurality) or order of winners and losers (STV - along with the type of election, the number of ballots, the number of seats, and the number of candidates. All of these variables are either variables that are kept track of within the election classes OR in the case of the type of election, this can be discerned by the type of election class. - This will be displayed onto the user's command line terminal interface, and only in the event the election is done without any errors. - The functions getName(), getAssignedBallots(), and getBallotNum() from the Candidate class will retrieve information from each Candidate to provide the information of the candidates in the election. - The functions getBallotCount() and getCandidateCount() will be used from the Ballots class to obtain information about the ballots themselves.	UC_012	Display Election Results

8. APPENDICES

No appendix is included in this document. Not applicable.